

UCP PARKING CONTROL

Smart Management System

Universidad Católica de Pereira

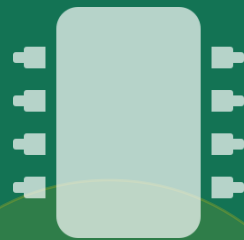
Proyecto de Asignatura Bases de Datos 2-Desarrollo Web · 2026

Daniel Bedoya y Santiago Betancur

ESP32 + IoT

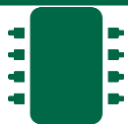
Node.js + React

MongoDB Atlas



¿Qué es UCP Parking Control?

Sistema IoT para la gestión automatizada del flujo vehicular en la Universidad Católica de Pereira. Utiliza un ESP32 con dos sensores infrarrojos en carriles separados — uno para entradas y otro para salidas — calculando los espacios disponibles en tiempo real y mostrándolos en un dashboard web.



Hardware IoT

ESP32 + 2 IR en carriles independientes: uno cuenta entradas, el otro salidas



Backend

API REST en Node.js + Express
almacena eventos en MongoDB Atlas



Frontend React

Dashboard SPA con contadores en tiempo real sin recargar la página

Arquitectura de la Maqueta



Fórmula de disponibilidad

$$\text{Espacios libres} = \text{Total celdas} - \text{Entradas} + \text{Salidas}$$

Cada carril tiene su propio sensor IR — sin lógica de secuencia, sin ambigüedad

Código del ESP32

Un sensor por carril — sin lógica de secuencia

```
#define PIN_ENTRADA 18
#define PIN_SALIDA 19

int entradas = 0, salidas = 0;
bool prevE = HIGH, prevS = HIGH;

void loop() {
    bool E = digitalRead(PIN_ENTRADA);
    bool S = digitalRead(PIN_SALIDA);

    // Flanco de bajada = vehículo cruzó el sensor
    if (E == LOW && prevE == HIGH) {
        entradas++; // +1 ENTRADA
        enviarEvento(entrada);
    }
    if (S == LOW && prevS == HIGH) {
        salidas++; // +1 SALIDA
        enviarEvento(salida);
    }
    prevE = E; prevS = S;
    delay(50);
}
```

01

Leer cada carril por separado

PIN_ENTRADA y PIN_SALIDA son pines independientes. Sin secuencia, sin ambigüedad.

02

Detectar flanco de bajada

Solo se cuenta cuando el sensor pasa de HIGH a LOW — el momento exacto en que el vehículo cruza.

03

Registros independientes

Entrada y salida son eventos separados. Cada uno incrementa su propio contador.

04

Enviar HTTP POST por Wi-Fi

El ESP32 envía tipo, contadores y espacios libres al endpoint /api/eventos sin cable.

Requisitos Funcionales

Los 15 requisitos definidos se agrupan en 5 módulos operativos

RF 01-05

Gestión de Usuarios y Vehículos

CRUD completo de conductores vinculados a carnetID y sus vehículos (placa, tipo).

RF 06

Control de Slots

Crear, actualizar y dar de baja celdas; cambiar estado a activo o en mantenimiento.

RF 07-10

Detección IoT

ESP32 lee el sensor de entrada y el de salida por separado, calcula libres y envía HTTP POST.

RF 11-13

Auditoría y Dashboard

Historial en MongoDB, filtros por fecha, y dashboard React con actualizaciones en vivo.

RF 14-15

Configuración y Robustez

Categorías de acceso por tipo y recuperación de estado al reiniciar el ESP32.

Requisitos No Funcionales

RNF-01

Disponibilidad

Alta

95% uptime en horario académico (6am–10pm)

RNF-02

Rendimiento

Alta

Backend responde en < 500ms al ESP32

RNF-04

Seguridad

Alta

Rutas POST/DELETE requieren autenticación

RNF-06

Mantenibilidad

Media

Patrón MVC en el backend (Node.js)

RNF-07

Portabilidad

Media

Frontend compatible con Chrome, Firefox, Edge

RNF-08

Tolerancia

Alta

ESP32 reintenta Wi-Fi cada 10s sin perder lectura

RNF-09

Precisión

Alta

Tasa de error en detección IR < 5%

RNF-10

Consistencia

Alta

Estado sincronizado entre ESP32 y MongoDB

Historias de Usuario

Las 10 historias definidas — selección de las más representativas

HU-01

Como Administrador

quiero **Ver en tiempo real los espacios libres** para *Tomar decisiones sin revisar físicamente el parqueadero*

HU-05

Como Administrador

quiero **Que los sensores cuenten entradas y salidas por carriles separados** para *No necesitar personal humano en el control de acceso*

HU-03

Como Estudiante

quiero **Consultar espacios disponibles desde mi celular** para *No desplazarme si el parqueadero está lleno*

HU-07

Como Administrador

quiero **Que el sistema recupere el conteo si el ESP32 se reinicia** para *No perder el registro acumulado durante el día*

HU-09

Como Admin. sistema

quiero **Que la API rechace solicitudes sin autenticación** para *Proteger los datos contra registros no autorizados*

Stack Tecnológico



Hardware

ESP32 + 2 Sensores Infrarrojos

Un sensor por carril · GPIO digital · Wi-Fi integrado · Firmware en C++



Backend

Node.js + Express

API REST · Patrón MVC · Endpoints: /api/eventos /api/slots /api/usuarios



Base de datos

MongoDB Atlas

No relacional · Colecciones: vehiculos · slots · eventos_parquadero



Frontend

React (SPA)

Single Page App · Actualizaciones en tiempo real · Responsive design



Comunicación

HTTP POST · Wi-Fi

ESP32 → Backend: JSON por Wi-Fi · Sin cable USB · Polling desde React

Conclusiones



Con 2 carriles independientes, la detección es directa: un sensor cuenta entradas y el otro salidas — sin lógica de secuencia ni riesgo de error por dirección.



El sistema con solo un ESP32 y 2 sensores IR es suficiente para gestionar el flujo vehicular de una maqueta universitaria funcional.



El stack Node.js + MongoDB Atlas + React cumple los 15 requisitos funcionales y los 10 no funcionales definidos en el proyecto.



El sistema es escalable: agregar más celdas solo implica actualizar el total en la configuración, sin modificar el hardware.

¡Gracias!

UCP Parking Control — Smart Management System

Universidad Católica de Pereira · Proyecto de Asignatura · 2026

ESP32

Node.js

MongoDB Atlas

React

Sensores IR

IoT